

Pearls AQI Predictor

End-to-end serverless Air Quality Index prediction system

Project Report

Prepared for the AQI prediction project submission

1. Executive Summary

Pearls AQI Predictor forecasts AQI 24, 48, and 72 hours ahead for eight major Pakistani cities. The system combines weather and pollutant observations, feature engineering, supervised model training, local Parquet storage with optional Hopsworks integration, GitHub Actions automation, and a Streamlit dashboard.

The deployment design is deliberately resilient: the dashboard can run from the checked-in feature cache and champion model without API keys, while scheduled workflows can enrich the cache with live data when secrets are configured.

2. System Architecture

Raw weather and air-pollution data are fetched from OpenWeather for current observations. Historical pollutant observations are obtained from OpenAQ. The feature pipeline computes EPA-style AQI, calendar features, lag features, rolling statistics, and AQI change rates. Features are written to the local Parquet cache and, when explicitly enabled, to Hopsworks.

The training pipeline creates multi-output targets for 24h, 48h, and 72h horizons, compares Ridge Regression, Random Forest, XGBoost, and optionally TensorFlow, and records metrics in the model registry. Streamlit loads the champion model and latest city row to render forecasts, history, alerts, model comparisons, and optional SHAP explanations.

3. Feature Pipeline

The engineered schema includes hour, day, month, day of week, weekend flag, day of year, temperature, humidity, pressure, wind speed and direction, cloud cover, PM2.5, PM10, NO2, SO2, CO, O3, AQI lags at 1h/24h/72h, 24-hour rolling mean and standard deviation, and AQI change rates at 1h/24h.

Feature imputation is performed within each city so one city's history cannot fill another city's lag or rolling values. This is important for a multi-city time-series dataset.

4. Historical Backfill

The backfill workflow runs the feature-generation path over a configurable historical window and writes the resulting training rows to the feature store. The default project configuration supports up to four years, subject to OpenAQ station coverage and API limits.

5. Training Pipeline and Evaluation

The model-selection process uses a chronological train/test split to reduce leakage from future observations. Candidate models are evaluated using RMSE, MAE, and R2. The best candidate is stored as the champion and its feature schema is recorded with the artifact.

Champion model	random_forest
RMSE	34.154
MAE	25.961
R2	0.644
Forecast horizons	24h, 48h, 72h
Training rows	52980
Test rows	13245

Candidate comparison

Model	RMSE	MAE	R2
Ridge	38.374	28.741	0.551
Random Forest	34.154	25.961	0.644

Champion performance by horizon

Horizon	RMSE	MAE	R2
24h	32.419	24.113	0.675
48h	34.497	26.357	0.637
72h	35.475	27.413	0.621

6. Automation and Deployment

GitHub Actions provides four operational workflows: CI tests on pushes and pull requests, a manual bootstrap for historical backfill and initial training, an hourly feature pipeline, and daily model training. The workflows default to the local cache unless the `USE_HOPSWORKS` repository variable is explicitly set to true, making the default path reliable without an external feature-store account.

Streamlit Community Cloud should be configured with `app/streamlit_app.py` as the main file and a supported modern Python runtime. The checked-in `data/features/aqi_features.parquet`, `models/registry.json`, and champion artifact allow the dashboard to open before the first live pipeline run. API keys belong in Streamlit Secrets or GitHub Actions Secrets and are never embedded in source code.

Operational workflow

Workflow	Cadence	Responsibility
CI test suite	Push / pull request	Compile code and run unit/integration tests
Feature pipeline	Every hour	Fetch observations, engineer features, update cache
Training pipeline	Daily	Compare candidates, promote champion, prune artifacts

Workflow	Cadence	Responsibility
Bootstrap pipeline	Manual	Backfill history and create the first model

7. Feature Inventory

The feature design combines short-term persistence, seasonal context, atmospheric conditions, and pollutant measurements. This gives the model both the current state of the city and signals that help it distinguish recurring daily patterns from sudden changes.

Group	Features	Purpose
Time	hour, day, month, day_of_week, is_weekend, day_of_year	Capture daily and seasonal cycles
Weather	temp, humidity, pressure, wind_speed, wind_deg, rel_humid	Represent dispersion and atmospheric conditions
Pollutants	PM2.5, PM10, NO2, SO2, CO, O3	Measure the current pollution mixture
Dynamics	AQI lags, rolling mean/std, change rates	Capture persistence, volatility, and direction

8. Dashboard and Explainability

The dashboard is organized around five user questions: What is the current AQI? What will happen over the next three days? What patterns are visible in the historical data? Why did the model make this forecast? Which candidate model won? The forecast view uses a gauge, KPI cards, colored AQI bands, and a compact timeline. The EDA view adds an AQI distribution and an hour-by-weekday heatmap. The explainability view provides global feature importance and signed current-row contributions using SHAP. The Model Lab compares candidate metrics and shows the champion's horizon-level errors.

SHAP is loaded only when the explainability tab is opened, its calculations are cached, and the chart input is bounded. This keeps the main dashboard responsive while preserving model transparency for users who need it.

9. Data Quality and Reliability

The local Parquet cache is the deployment safety net. Feature writes are de-duplicated by city and timestamp, and Hopsworks is optional. Derived imputation is performed within each city to prevent cross-city leakage. Training targets are joined by timestamp offsets so missing observations do not silently turn a 24-row shift into a false 24-hour horizon.

10. Limitations and Future Work

Forecast quality depends on station coverage, API freshness, and the availability of complete hourly observations. Timestamp-based target joins correctly omit unavailable future observations, but longer-term production work should add explicit hourly resampling and missingness indicators. Future work should add calibrated uncertainty intervals, richer station-level spatial features, drift monitoring, and a dedicated alert delivery channel.

11. Reproduction Checklist

1. Configure OPENWEATHER_API_KEY and optionally OPENAQ_API_KEY and HOPSWORKS_API_KEY.
2. Run the bootstrap workflow once.
3. Confirm the CI workflow passes.
4. Deploy app/streamlit_app.py on Streamlit Community Cloud using Python 3.11.
5. Run the hourly and daily workflows manually once to verify repository write permissions.